React Routing & Patterns Assessment

Oct 28, 2026 · @Abdulrahman Ibrahim

Instructions

This assessment has two parts: 10 multiple-choice questions (Part 1) and a coding task (Part 2) built with Vite, React, TypeScript and React Router.

- **Part 1:** choose one answer per question. Write your answers in a file called `ANSWERS.md` at the root of your Part 2 repo (format: `1. B`).
- **Part 2:** build the Project Portal app described below and submit it as a **public GitHub repository**.
- **Starter files:** two files are provided and must be copied into your project **without modification**. They will be checked against the originals.
- **Follow-up interview:** after submission you will walk your mentor through your code, explain your decisions, and make small changes to it live. Write only code you can explain.

Part 1: Multiple-choice questions

All snippets assume React 18+, React Router 6.4+ (or 7) in declarative mode, and TypeScript with `strict: true`.

Q1. Where HOCs are created

```
function Page() {
  const Guarded = withAuth(Profile);
  return <Guarded />;
}
```

`Profile` has a text input backed by `useState`. The user types into it, then `Page` re-renders for an unrelated reason. What happens?

- A) Nothing unusual; the typed text is kept
- B) TypeScript reports an error
- C) The typed text is lost, because `Guarded` is a new component type on every render
- D) React throws an infinite-loop error

Q2. Active links

The current URL is `/dashboard/settings`. The sidebar contains:

```
<NavLink to="/dashboard">Overview</NavLink>
```

Is the Overview link marked active?

- A) Yes, because `/dashboard` matches as a parent of the current URL
- B) No, because `NavLink` only matches exact paths
- C) Only if Overview is an index route
- D) Only if `caseSensitive` is set

Q3. Nested layouts

```
<Route path="/dashboard" element={<DashboardLayout />}>
  <Route path="projects" element={<ProjectList />} />
</Route>

function DashboardLayout() {
  return (
    <div>
      <Sidebar />
      <main>
        <h1>Dashboard</h1>
      </main>
    </div>
  );
}
```

What appears at `/dashboard/projects`?

- A) Only `ProjectList`; the layout is replaced
- B) The sidebar and heading, but not `ProjectList`
- C) A 404 page
- D) An error is thrown

Q4. Route parameters

In this snippet, `Project.id` is a `number`:

```
const { projectId } = useParams();
const project = projects.find((p) => p.id === projectId);
```

What happens?

- A) The matching project is found
- B) `project` is always `undefined`, with no error
- C) It throws at runtime
- D) TypeScript reports an error, because `number` and `string | undefined` have no overlap

Q5. Search parameters

The current URL is `/projects?status=active&q=web`. A handler runs:

```
setSearchParams({ q: 'api' });
```

What is the URL afterwards?

- A) `/projects?status=active&q=api`
- B) `/projects?q=api`
- C) `/projects?status=active&q=web&q=api`
- D) Unchanged until the page reloads

Q6. Refs and rendering

```
function ClickCounter() {
  const clicks = useRef(0);
  return (
    <button onClick={() => { clicks.current += 1; }}>
```

```
    Clicked {clicks.current} times
  </button>
 );
}
```

The user clicks the button three times. What does the label show?

- A) `Clicked 0 times`
- B) `Clicked 3 times`
- C) `Clicked 1 times`
- D) Nothing; React throws because refs cannot be read during render

Q7. Typing a HOC

```
function withLoading<P extends object>(Component: ComponentType<P>) {
 return function WithLoading({ isLoading, ...rest }: P & { isLoading: boolean }) {
  if (isLoading) return <p>Loading...</p>;
  return <Component {...rest} />;
 };
}
```

What happens when you compile this?

- A) It compiles and works as intended
- B) TypeScript reports an error on `<Component {...rest} />`, because `rest` is not assignable to `P`
- C) It compiles, but `isLoading` still reaches `Component`
- D) It throws at runtime

Q8. Navigating from code

```
function Account() {
 const { user } = useAuth();
 const navigate = useNavigate();
 if (!user) navigate('/login');
 return <Profile />;
}
```

What is wrong with this redirect?

- A) Nothing; this is the standard pattern
- B) `useNavigate` only works in class components
- C) `navigate` is called during render; it belongs in an effect or event handler, or the component should render `<Navigate to="/login" />`
- D) TypeScript reports an error because `navigate` returns a promise

Q9. Aborting requests

```
useEffect(() => {
 const controller = new AbortController();
 fetch(`/api/projects/${projectId}`, { signal: controller.signal })
  .then((res) => res.json())
  .then(setProject)
  .catch(setError);
 return () => controller.abort();
}, [projectId]);
```

`projectId` changes from `p1` to `p2` before the first response arrives. What goes wrong?

- A) Both responses are written to state
- B) The request for `p2` is never sent
- C) TypeScript reports an error on `signal`
- D) The `p1` request rejects with an `AbortError`, so `.catch(setError)` shows an error even though nothing failed

Q10. When state updates

```
const [count, setCount] = useState(0);

function handleClick() {
  setCount(5);
  console.log(count);
}
```

What does the first click log?

- A) `0`
- B) `5`
- C) `undefined`
- D) `5`, but only after the re-render finishes

Part 2: Project Portal

Build a multi-page portal where team members sign in, browse projects and open project details, and admins can reach a settings page. Pages share layouts through nested routes, and access control and loading screens are added with higher-order components (HOCs). Data comes from a provided mock API that is slow and fails about 20% of the time.

1. Setup and constraints

- Scaffold with Vite's `react-ts` template. Keep `"strict": true` in `tsconfig`.
- Use `react-router-dom` (6.4+ or 7) in declarative mode: `<BrowserRouter>` and `<Routes>`. Do not use route loaders or actions.
- Use React Context for the signed-in user. Do **not** use Redux, Zustand, React Query or any other state library.
- No UI component libraries. Plain CSS or CSS Modules only.
- No `any`, no `as unknown as`, no `// @ts-ignore` or `// @ts-expect-error`.
- `npm run build` must finish with zero TypeScript errors.
- Copy the two starter files below into `src/` **unmodified**.

2. Authentication

- Create an `AuthProvider` and a `useAuth()` hook that throws a clear error when used outside the provider.
- The login page has a controlled username input and calls `login()` from the mock API. Show the error message on failure and disable the button while the request is in flight.

- Keep the session in `localStorage` and restore it on reload. Handle a missing or corrupted stored value without crashing.
- The header shows the signed-in user's name and a Log out button.

3. Higher-order components

- `withAuth(Component, options?)`: when nobody is signed in, redirect to `/login` with `<Navigate>`, remembering the page that was requested. When `options.role` is set and the user's role does not match, show "You do not have access to this page."
- `withLoading(Component)`: accepts an extra `isLoading: boolean` prop and shows a loading indicator while it is true. `isLoading` must **not** reach the wrapped component.
- Type both HOCs with generics so the wrapped component's own props stay fully typed.
- Give each HOC result a `displayName` such as `withAuth(SettingsPage)`.
- Create every wrapped component **once, at module level**, never inside another component.
- Wrapped page components contain no auth checks and no loading logic.

4. Routes and layouts

Implement this route tree:

```
/                    Home
/login               Login
/dashboard           Dashboard layout (signed-in users only)
  (index)            Overview
  projects           Project list
  projects/:projectId    Project details
  settings           Settings (admins only)
*                    Not found
```

- A root layout with the header (Home and Dashboard links, plus user name and Log out) is shown on every page.
- The dashboard layout shows a sidebar (Overview, Projects, Settings) on every dashboard page. Child pages render through `<Outlet />`.
- Highlight the active link in both the header and the sidebar. Overview and Home must not stay active on child pages.
- After signing in, return the user to the page they originally requested, or `/dashboard` if there was none. A signed-in user who opens `/login` is sent to `/dashboard`.
- Load the Settings page with `React.lazy` and `Suspense`, with a visible fallback.

5. Projects

- **Project list:** load projects from the mock API. Show exactly one of three UI states: loading (through `withLoading`), error (with the message and a **Retry** button), or ready.
- Filter by status (dropdown) and by name (search box). Store both in the URL with `useSearchParams` (for example `?status=active&q=web`), so a refresh or a shared link keeps them. Changing one filter must keep the other.
- **Project details:** read `projectId` with `useParams` and load the project. Show "Project not found" when the API returns `null`. Show the name, description, status, owner, and start and due dates formatted with `Intl.DateTimeFormat`.

- Responses that arrive after the user has moved to another project, or left the page, must be ignored. Use the effect's cleanup.
- Add a **Back** button. Go back in history when the user arrived from inside the app; otherwise go to `/dashboard/projects`.
- **Overview:** show the number of projects per status.
- **Settings:** a "Compact list" toggle, saved to `localStorage`, that changes how the project list is displayed.

6. Tests

- Add at least two tests with Vitest and React Testing Library, using `MemoryRouter`: one showing that an unknown URL renders the Not found page, and one showing that `withAuth` sends a signed-out user to `/login`.

7. NOTES.md

In your own words, 1–3 short paragraphs each:

1. Why the HOCs must be applied at module level, and what breaks if they are created inside a component.
2. How your `withLoading` types keep `isLoading` from reaching the wrapped component.
3. How returning to the originally requested page after login works, step by step.
4. One thing you would change with more time.

Starter file 1: `src/types.ts`

```
export type Role = 'admin' | 'member';
export type ProjectStatus = 'active' | 'on_hold' | 'completed';

export interface User {
  id: string;
  username: string;
  name: string;
  role: Role;
}

export interface Project {
  id: string;
  name: string;
  description: string;
  status: ProjectStatus;
  ownerName: string;
  startDate: string; // YYYY-MM-DD
  dueDate: string; // YYYY-MM-DD
}
```

Starter file 2: `src/api/mockApi.ts`

```
import type { Project, User } from '../types';

const LATENCY_MS = 700;
const FAILURE_RATE = 0.2;
```

```
const users: User[] = [
  { id: 'u1', username: 'sara', name: 'Sara Mahmoud', role: 'admin' },
  { id: 'u2', username: 'youssef', name: 'Youssef Adel', role: 'member' },
];

const projects: Project[] = [
  { id: 'p1', name: 'Customer Mobile App', description: 'Native app for account management
and payments.', status: 'active', ownerName: 'Sara Mahmoud', startDate: '2026-03-01',
dueDate: '2026-12-15' },
  { id: 'p2', name: 'Billing Migration', description: 'Move invoicing to the new billing
platform.', status: 'on_hold', ownerName: 'Youssef Adel', startDate: '2026-05-10', dueDate:
'2027-02-28' },
  { id: 'p3', name: 'Web Portal Redesign', description: 'Refresh the self-service web portal.',
status: 'active', ownerName: 'Sara Mahmoud', startDate: '2026-07-01', dueDate: '2027-01-
31' },
  { id: 'p4', name: 'Support Chatbot', description: 'First-line chatbot for common support
questions.', status: 'completed', ownerName: 'Youssef Adel', startDate: '2026-01-15',
dueDate: '2026-09-30' },
  { id: 'p5', name: 'Analytics Dashboard', description: 'Internal dashboard for product usage
metrics.', status: 'active', ownerName: 'Youssef Adel', startDate: '2026-08-20', dueDate:
'2027-03-31' },
];

function simulate<T>(fn: () => T): Promise<T> {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (Math.random() < FAILURE_RATE) {
        reject(new Error('Network error: please try again.'));
        return;
      }
      try {
        resolve(fn());
      } catch (err) {
        reject(err);
      }
    }, LATENCY_MS);
  });
}

export function login(username: string): Promise<User> {
  return simulate(() => {
    const user = users.find((u) => u.username === username.trim().toLowerCase());
    if (!user) throw new Error('Unknown username.');
    return { ...user };
  });
}

export function fetchProjects(): Promise<Project[]> {
  return simulate(() => projects.map((p) => ({ ...p })));
}

export function fetchProject(id: string): Promise<Project | null> {
  return simulate(() => {
    const project = projects.find((p) => p.id === id);
    return project ? { ...project } : null;
```

```
  });
}
```

Evaluation and submission

Your submission is scored on the areas below, then confirmed in a follow-up interview where you explain and change your code live.

Area	What we look at	Weight
Part 1 MCQs	Correct answers in ANSWERS.md	20%
Functionality	Every requirement in sections 2 to 5 works, including redirects, Retry and URL filters	25%
Routing and architecture	Route tree, layouts, HOC design, auth context, state in the right place	20%
TypeScript	Strict mode, generic HOC typing, no escape hatches	15%
Code quality	Readable components, effect cleanup, accessible forms and navigation	10%
Tests and NOTES.md	Tests pass and cover the right logic; notes are clear and in your own words	10%

Before you submit

- ☐ Repo is public and has a README with install and run steps
- ☐ ANSWERS.md contains all 10 answers
- ☐ src/types.ts and src/api/mockApi.ts are unchanged from this brief
- ☐ npm run build and npm test both pass
- ☐ No any, @ts-ignore or @ts-expect-error anywhere in src/
- ☐ Login redirect-back, admin-only Settings and the 404 page all tested by hand
- ☐ The session and the list filters survive a page reload
- ☐ NOTES.md answers all four questions